CLOUD-BASED PHARMACY INVENTORY MANAGEMENT SYSTEM WITH CI/CD PIPELINE AND MACHINE LEARNING PREDICTIONS

BABIYASHINI VARADARAJ

STUDENT ID: LU202401010001

THESIS SUBMITTED IN FULFILMENT OF THE DEGREE OF BACHELOR(HONS) IN INFORMATION TECHNOLOGY (BIT)

FACULTY OF COMPUTER SCIENCE & MULTIMEDIA

LINCOLN UNIVERSITY

MALAYSIA

JULY 2026

DECLARATION

I hereby declare that this thesis entitled “Cloud based pharmacy inventory management system with ci/cd pipeline and machine learning predictions” is my original work and has been prepared in partial fulfilment of the requirements for the award of the Bachelor of Information Technology (BIT) degree at Lincoln University College.

I declare that this thesis has not been submitted previously, either in whole or in part, to any other university, college, or institution for the purpose of obtaining any academic qualification. All sources of information, ideas, data, and other contributions obtained from the work of others have been duly acknowledged and appropriately referenced.

I further confirm that, except where otherwise acknowledged, the work presented in this thesis is my own work and represents the results of my research and study**.**

Candidate’s Name: Babiyashini Varadaraj

Student ID / Registration No.: LU202401010001

Signature: _____

Date: _____

SUPERVISOR'S APPROVAL

I hereby certify that this thesis entitled "Cloud based pharmacy inventory management system with ci/cd pipeline and machine learning predictions", prepared and submitted by Babiyashini Varadaraj, bearing Student ID/Registration No. LU202401010001, in partial fulfilment of the requirements for the award of the Bachelor of Information Technology (BIT) degree at Lincoln University College Malaysia, has been carried out under my supervision.

To the best of my knowledge, the work presented in this thesis is the candidate's own work and meets the academic requirements expected of an undergraduate thesis. I therefore recommend that this thesis be submitted for evaluation and consideration for the award of the Bachelor of Information Technology degree.

Supervisor's name: Mr. R.A.G. Ranawaka

Signature: _____

Date: _____

Designation/department: _____

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to God for giving me the strength, wisdom and guidance to complete this project successfully. I would also like to express my sincere gratitude to my supervisor, Mr. Gamini Ranwaka, for his guidance, feedback and support throughout the development of this project. I also thank the pharmacy owners and staff members who provided information about their existing inventory practices and participated in the evaluation of the system. Their practical feedback helped identify the main problems addressed by the project. I am also grateful to my lecturers, colleagues, friends and family members for their encouragement and support throughout the project.

ABSTRACT

The Cloud-Based Pharmacy Inventory Management System is a web application addressing stockouts, overstocking, expiry losses, and manual record-keeping in small and medium pharmacies. It provides user, medicine, supplier, sales, invoice, reporting, expiry tracking, and low-stock monitoring, with a machine-learning component for short-term demand prediction using Random Forest Regression. Built with Python Flask, MySQL, HTML/CSS/Bootstrap/Jinja2, Docker, and AWS (EC2, RDS, S3, VPC). Terraform manages Infrastructure as Code, GitHub Actions handles CI/CD, and an auto-healing Bash script restarts the application on failure. HTTPS uses Nginx and Let's Encrypt. The project followed Design Science Research and Agile Scrum. Testing included 20 unit tests, 8 integration tests, and 5 system-level scenarios, all passed. UAT with five pharmacy staff gave 4.6/5 satisfaction. Performance: 1.8s dashboard load, 25s deployment, 35s auto-heal recovery. The ML component uses Random Forest Regression (80:20 split, 100 estimators, depth 10), evaluated via MAE, RMSE, and R^2 (approximately 0.73). REST API integration is future work, referencing Microsoft REST API guidance, HTTP semantics, OpenAI, and OWASP API Security standards.

Keywords: pharmacy inventory management, cloud computing, machine learning, Random Forest Regression, Docker, AWS, CI/CD, GitHub Actions, Terraform, REST API, Open API, API security.

TABLE OF CONTENTS

DECLARATION ii

SUPERVISOR'S APPROVAL iii

ACKNOWLEDGEMENT iv

ABSTRACT v

TABLE OF CONTENTS vi

LIST OF FIGURES xi

LIST OF ABBREVIATIONS xiii

CHAPTER ONE: INTRODUCTION 1

1.1 Background of the Study 1

1.2 Problem Statement 2

1.3 Project Objectives 3

1.4 Scope and Limitations 3

1.5 Significance of the Study 4

1.6 Report Structure 5

CHAPTER TWO: LITERATURE REVIEW 6

2.1 Introduction 6

2.2 Pharmacy Inventory Management 6

2.3 Inventory Management Approaches 7

2.4 Machine Learning for Demand Forecasting 8

2.5 Cloud Computing 9

2.6 Docker Containerization 10

2.7 CI/CD and DevOps 11

2.8 Infrastructure as Code 12

2.9 REST APIs and Microsoft API Design Guidance 13

2.10 Open API 14

2.11 API Security 15

2.12 Research Gap 16

2.13 Chapter Summary 17

CHAPTER THREE: METHODOLOGY 19

3.1 Research Methodology 19

3.2 Agile Scrum 19

3.3 Data Collection 20

3.4 Machine Learning Approach 21

3.5 Ethical Considerations 22

3.6 Chapter Summary 22

CHAPTER FOUR: SYSTEM ANALYSIS AND DESIGN 24

4.1 Requirements Analysis 24

4.2 System Architecture 26

4.3 Chapter Summary 32

CHAPTER FIVE: IMPLEMENTATION 33

5.1 Development Environment 33

5.2 AWS infrastructure setup (terraform) 34

5.3 User Authentication 34

5.4 Module Implementation 35

5.5 Medicine Management 36

5.6 Sales and Invoicing 36

5.7 Supplier Management 36

5.8 Reports and Dashboard 37

5.9 Machine Learning 37

5.10 Docker file 38

5.10.1 building and running 39

5.11 Docker 39

5.12 CI/CD Pipeline 40

5.12.1 CI/CD pipeline implementation 40

5.13 Infrastructure as Code 41

5.13.1 AWS infrastructure setup (terraform) 41

5.14 Auto-Healing 43

5.15 HTTPS 44

5.16 Future REST API Standard 44

CHAPTER SIX: TESTING AND RESULTS 46

6.1 Testing Strategy 46

6.2 Performance Evaluation 49

CHAPTER SEVEN : DISCUSSION 54

7.1 Interpretation of Results 54

7.2 Comparison with Existing Systems 55

7.3 Challenges and Solutions 55

7.4 Limitations 56

CHAPTER EIGHT: CONCLUSION AND FUTURE WORK 58

8.1 Conclusion 58

8.2 Future Work 59

8.3 Personal Reflection 60

REFERENCES 61

APPENDICES 65

Appendix A: Project Objectives Checklist 65

Appendix B: Interview Questions and Findings 65

Appendix C : Requirements Traceability 66

Appendix D: Agile Scrum Sprint Plan 67

Appendix E: Machine Learning Dataset 67

Appendix F: AWS and Terraform 68

Appendix G: Research Methodology..... 68

Appendix H: ML Model Evaluation 68

LIST OF FIGURES

Figure 1: System Architecture 27

Figure 2: Use Case Diagram 28

Figure 3 : System's database diagram 30

Figure 4: Login page wireframe 31

Figure 5: Inventory Dashboard wireframe 31

Figure 6 : Create virtual environment 33

Figure 7 : The default 8gb was insufficient for docker builds 34

Figure 8: User Authentication module 35

Figure 9 : Docker Implementation 38

Figure 10 : GitHub Actions workflow and 40

Figure 11: Terraform Infrsfrasructure 41

Figure 12 : User Authentication module 42

LIST OF TABLES

Table 1 : Tools and Technologies 21

Table 2 : Functional Requirement 25

Table 3 : Non-Functional Requirements 26

Table 4 : Relationships between entities 29

Table 5 : API Design Reference. 45

Table 6 : Unit Testing 46

Table 7 : Integration Testing 47

Table 8 : System Testing 48

Table 9 : User Acceptance Testing 49

Table 10 : Performance Evaluation 50

Table 11 : Testing strategy 51

Table 12 : Unit Testing Results 53

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
EC2	Elastic Compute Cloud
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure as Code
IETF	Internet Engineering Task Force
ML	Machine Learning
MySQL	My Structured Query Language
RDS	Relational Database Service
REST	Representational State Transfer
R^2	Coefficient of Determination
S3	Simple Storage Service
SSL	Secure Sockets Layer
UAT	User Acceptance Testing
VPC	Virtual Private Cloud

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

The healthcare sector depends on reliable processes for storing, managing and dispensing medicines. Pharmacies must maintain sufficient stock while avoiding unnecessary accumulation of products that may expire before sale. Medication-related processes can create safety risks when information is incomplete, outdated or difficult to access. The World Health Organization (2016) identifies medication errors as an important patient-safety concern and highlights the need to understand their causes and develop strategies to reduce them. Electronic tools can also support safer primary care by improving access to information and supporting healthcare processes (World Health Organization, 2016).

Despite the availability of digital technologies, small and medium-sized pharmacies may still rely on paper records or basic spreadsheet-based processes. Such approaches can make it difficult to maintain accurate stock quantities, identify medicines approaching expiry, review sales trends and make timely purchasing decisions.

The investigation for this project involved interviews with five local pharmacy owners and staff members. The interview findings identified medicine expiry losses, stockouts and

overstocking, time-consuming manual processes, deployment difficulties and security weaknesses as key problems.

The proposed system modernizes these processes through a web-based inventory application. It provides role-based authentication, medicine CRUD operations, expiry and low-stock monitoring, sales recording, invoice generation, supplier management, reporting and machine-learning-based demand prediction. The system is also deployed using cloud infrastructure and supported by Docker, Terraform and GitHub Actions so that the project demonstrates both application development and cloud/DevOps engineering practices.

1.2 Problem Statement

The project investigation identified five main problem areas. First, pharmacies reported losses associated with medicine expiry and lacked automated alerts for approaching expiry dates. Second, stockouts and overstocking occurred because purchasing decisions were not supported by demand forecasting.

Third, daily stock counting, paper-based sales recording and monthly reporting were time-consuming and prone to manual errors. Fourth, system deployment required manual server access, making updates slower and increasing the possibility of deployment mistakes. Fifth, manual systems often lacked user authentication, encryption and auditability.

1.3 Project Objectives

The general objective is to design, develop and deploy a secure cloud-based pharmacy inventory management system with integrated machine learning for demand prediction and an automated CI/CD pipeline.

- To analyze existing pharmacy inventory processes and identify system requirements.
- To develop a web-based system for medicine tracking, sales recording, supplier management and expiry monitoring.
- To implement a machine-learning model for short-term demand prediction using historical sales data.
- To containerize the application using Docker and deploy it on AWS.
- To implement an automated CI/CD pipeline using GitHub Actions.
- To evaluate system functionality, performance, security and user satisfaction.

1.4 Scope and Limitations

The project scope includes user management, medicine management, sales management, supplier management, reports, machine-learning demand prediction, Docker

containerization, AWS infrastructure, HTTPS security and auto-healing. The system supports Admin, Pharmacist and Cashier roles. It is designed for a single pharmacy and is web-based.

The project does not include a native mobile application, barcode scanner integration, payment gateway integration, multi-branch support, SMS/email notifications or offline operation. The machine-learning component is limited by the synthetic nature and practical representativeness of the available historical sales data. The implemented training query uses approximately 12 months of historical sales information. A REST API is identified as future work rather than an implemented feature.

1.5 Significance of the Study

The system provides potential value to pharmacy owners by improving visibility of stock and sales information and supporting data-driven decisions. Pharmacy staff can perform sales and stock operations more quickly than with paper-based processes. Customers may benefit indirectly from better medicine availability and more efficient checkout. From an academic perspective, the project demonstrates the integration of a web application, machine learning, cloud infrastructure, Docker containerization, Infrastructure as Code and CI/CD automation in a practical business domain.

1.6 Report Structure

Chapter 1 introduces the project, its background, problem statement, objectives, scope and significance. Chapter 2 reviews relevant literature and technical standards. Chapter 3 explains the research and development methodology. Chapter 4 presents the requirements and system design. Chapter 5 describes implementation and deployment. Chapter 6 presents testing and results. Chapter 7 discusses the findings, challenges and limitations. Chapter 8 concludes the project and presents future work**.**

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the academic and technical concepts supporting the proposed pharmacy inventory management system. The review covers pharmacy inventory management, inventory control approaches, machine learning for demand forecasting, cloud computing, containerization, CI/CD, Infrastructure as Code, REST API design, Open API and API security. The purpose is to establish the theoretical basis for the design choices made in the project and identify gaps that the proposed system addresses.

2.2 Pharmacy Inventory Management

Pharmacy inventory management involves maintaining appropriate quantities of medicines while monitoring stock movement, purchasing, expiry and availability. Effective information management is important because medicine-related processes can affect both operational efficiency and patient safety.

The World Health Organization (2016) discusses medication errors and emphasizes investigation of their causes and strategies for reducing them. The use of electronic tools

is also identified as an area relevant to safer primary care. These perspectives support the need for reliable digital information systems within healthcare operations.

The project investigation found that the participating pharmacies experienced practical problems with expiry monitoring, stockouts, overstocking and manual records. The proposed system therefore focuses on providing a centralized inventory record, search functionality, low-stock alerts, expiry monitoring, sales history and reports. The system is intended to support pharmacy staff rather than replace professional decisions.

2.3 Inventory Management Approaches

Economic Order Quantity (EOQ) is a classical inventory model that attempts to identify an order quantity that balances ordering and holding costs. ABC analysis classifies inventory based on value or importance, allowing high-impact items to receive greater attention.

Just-in-Time (JIT) attempts to reduce inventory holding by receiving items close to the time they are required. These approaches provide useful theoretical foundations, but pharmaceutical inventory introduces additional considerations such as expiry dates, critical medicines and variable demand.

The proposed system therefore uses practical monitoring features such as minimum stock thresholds and expiry tracking rather than implementing a single classical inventory model as its complete decision-making mechanism.

2.4 Machine Learning for Demand Forecasting

Demand forecasting can support inventory decisions by estimating future product requirements from historical information. Machine-learning techniques can be used when historical sales data contains relationships or patterns that can be learned from examples. The proposed system uses Random Forest Regression for demand prediction.

The implementation creates temporal and sales-related features from historical transaction data, including day, week, month, quarter, weekend status, 7-day, 14-day and 30-day sales lags, 7-day and 30-day moving averages, medicine price and price ratio.

The implemented model uses an 80:20 train-test split and a RandomForestRegressor configured with 100 estimators and a maximum depth of 10. StandardScaler is also applied to the feature data. Model performance is evaluated using Mean Absolute Error (MAE), Root Mean Square Error (RMSE) and R^2.

The project reports an R^2 performance of approximately 0.73. R^2 should not be described as “73% accuracy”, because regression models are evaluated using regression metrics rather than classification accuracy.

The training query uses approximately 12 months of historical sales data. However, the dataset is synthetic and may not fully represent real pharmacy demand patterns such as seasonal variation, promotions, supplier delays or unusual purchasing behavior. Therefore, the model should be considered a decision-support component rather than a replacement for professional purchasing judgment.

The inventory insight functionality also calculates recent average daily sales over a 30-day period to estimate operational demand, stockout-related information and suggested order quantities. This calculation should be distinguished from the trained Random Forest model.

2.5 Cloud Computing

Cloud computing provides on-demand access to computing, storage, database and networking resources. In this project, AWS is used to host the application and database infrastructure. EC2 provides the application server, RDS provides MySQL database hosting, S3 provides object storage and VPC provides network isolation and configuration. Cloud deployment supports remote access and provides a foundation for future scalability.

2.6 Docker Containerization

Containerization provides a method for packaging an application together with its required software dependencies into an isolated and portable environment. Pahl (2015) describes containerization as an important technology within cloud and Platform as a Service environment because it can support application portability and deployment consistency.

Docker is used in this project to package the Flask-based pharmacy inventory application. The Docker image contains the application and the dependencies required to execute it. This reduces differences between development and deployment environments and makes the application easier to deploy on the cloud server.

Containerization is particularly useful when combined with CI/CD because a new application image can be created automatically when changes are introduced into the source code.

The resulting image can then be deployed to the target environment. In this project, Docker is therefore not considered as an isolated technology. It forms part of the deployment architecture together with AWS EC2 and GitHub Actions.

2.7 CI/CD and DevOps

Continuous Integration (CI) involves regularly integrating software changes into a shared codebase and automatically validating those changes. Continuous Delivery and Continuous Deployment extend this process by automating the preparation and release of software.

Shahin, Babar and Zhu (2017) conducted a systematic review of continuous integration, continuous delivery and continuous deployment and identified automation, tooling, testing and organizational practices as important aspects of implementing these approaches.

Humble and Farley (2010) describe continuous delivery as an approach that aims to make software releases more reliable through automated build, testing and deployment processes. Automation can reduce repetitive manual deployment activities and provide a more consistent release process.

The proposed system uses GitHub Actions to implement a CI/CD workflow. Changes pushed to the main branch trigger the workflow, which connects to the cloud server and performs the required deployment operations.

The deployment process includes obtaining the latest source code, building the Docker image, stopping the previous container and starting the updated application container. The project evaluation recorded an average deployment time of approximately 25 seconds. This

demonstrates the practical benefit of automation within the developed prototype by reducing the manual steps required to deploy application changes.

2.8 Infrastructure as Code

Infrastructure as Code (IaC) is an approach in which infrastructure configuration is defined using machine-readable configuration files rather than being created entirely through manual configuration.

Artac et al. (2017) describe Infrastructure as Code as an approach that brings software engineering principles into infrastructure management. IaC can improve repeatability, consistency and maintainability because infrastructure configuration can be stored and managed alongside application source code.

Terraform is used in the proposed system to define cloud infrastructure. The project uses Terraform for resources associated with the AWS environment, including infrastructure required for the application deployment.

The use of Terraform supports the reproducibility of the project's cloud environment and reduces dependence on manually repeating configuration steps. It also complements CI/CD

because infrastructure configuration can be maintained as part of the project's source-controlled development process.

2.9 REST APIs and Microsoft API Design Guidance

Representational State Transfer (REST) is an architectural style commonly used for web-based services. HTTP provides standard methods and semantics for communication between clients and servers. Fielding et al. (2022) provide the standard HTTP semantics used by modern web applications.

Microsoft's REST API design guidance recommends using appropriate HTTP methods, meaningful resource-oriented URLs, suitable status codes and consistent representations when designing RESTful APIs (Microsoft, 2026).

A REST API would be useful for the proposed pharmacy system because it could allow external applications, mobile applications or other pharmacy systems to interact with inventory and prediction functionality.

However, REST API functionality is considered a future extension of the current project rather than an implemented feature. The proposed design includes resources such as medicines, sales and demand predictions and is intended to demonstrate how the system

could later be extended to support external integrations. Therefore, the API design presented in this thesis should not be interpreted as evidence that a production REST API has already been implemented and tested.

2.10 Open API

Open API provides a standard format for describing HTTP APIs. The Open API Specification can be used to document available endpoints, request parameters, responses and other API characteristics (Open API Initiative, 2024).

For the proposed pharmacy inventory system, Open API could provide a formal contract for a future REST API. This would make the API easier for developers to understand and could support future integration with mobile applications or third-party systems.

The proposed future API includes resources such as medicines, sales and demand predictions. Open API documentation could describe the expected request and response structures for these resources.

Since the API is a future extension of the system, Open API is included as a design and documentation consideration rather than as evidence of an implemented API.

2.11 API Security

API security is important because APIs may expose application data and business functions to external clients. Poorly protected APIs can therefore create risks involving unauthorized access, data exposure and misuse of application functionality.

The OWASP API Security Top 10 (2023) identifies security risks including Broken Object Level Authorization, Broken Authentication, Broken Object Property Level Authorization, Unrestricted Resource Consumption, Broken Function Level Authorization and Security Misconfiguration (OWASP Foundation, 2023).

A future API for the pharmacy inventory system should therefore incorporate authentication, role-based authorization, object-level access controls, input validation, HTTPS, appropriate rate controls and secure configuration.

Security is especially important for a pharmacy inventory system because the application may contain sensitive operational information such as user accounts, medicine information, supplier records and sales information.

The current thesis therefore treats API security as part of the design requirements for future API development rather than claiming that a complete externally accessible API has already been implemented.

2.12 Research Gap

The literature demonstrates that different technologies can address different aspects of pharmacy inventory management. Conventional inventory management approaches provide established methods for stock classification, stock rotation and inventory control. Machine learning can provide additional support for demand forecasting, while cloud computing can support remote deployment and accessibility.

Similarly, Docker can improve application portability, CI/CD can automate software deployment, and Infrastructure as Code can improve infrastructure consistency and reproducibility. Existing research has examined these areas independently and has also investigated machine learning for inventory optimization. However, there remains a practical opportunity to integrate these capabilities into a single small-pharmacy-oriented information system. In particular, a system can combine core pharmacy inventory functions with short-term demand prediction and an automated cloud deployment environment.

The proposed project addresses this gap by integrating medicine and supplier management, sales recording, stock monitoring, expiry tracking, reporting and machine learning-based demand prediction within a cloud-deployed web application. Docker, AWS, GitHub Actions and Terraform are incorporated to support deployment and infrastructure management.

The project also identifies REST API interoperability as a future extension. This could allow external applications and mobile clients to interact with the pharmacy inventory system through a documented and secured interface. A further limitation identified by the literature and the current project is the availability of high-quality real pharmacy sales data. The developed model uses synthetic historical sales data; therefore, future research should evaluate the forecasting approach using larger and real-world pharmacy datasets.

2.13 Chapter Summary

This chapter reviewed the theoretical and technical concepts supporting the development of the Cloud-Based Pharmacy Inventory Management System. The review covered pharmacy inventory management, inventory classification and stock rotation, machine learning-based demand forecasting, cloud computing, Docker containerization, CI/CD, Infrastructure as Code, REST API design, Open API and API security.

The literature supports the use of structured inventory management practices for controlling pharmaceutical stock and reducing stock-related problems. Machine learning provides an additional opportunity to support demand forecasting, while Random Forest Regression is suitable for the regression-based prediction component developed in this project.

Cloud computing, Docker, CI/CD and Terraform provide complementary technologies for deploying and maintaining the application. REST, Open API and API security were reviewed as potential future extensions rather than implemented functionality.

Overall, the literature review supports the technical choices made in this project and identifies the integration of pharmacy inventory management, demand prediction and automated cloud deployment as the main contribution of the proposed system.

CHAPTER THREE: METHODOLOGY

3.1 Research Methodology

The project followed a Design Science Research (DSR) approach. Design Science is suitable for information systems projects that create and evaluate practical artifacts. The project process consisted of problem identification and motivation, definition of objectives, design and development, demonstration through deployment, and evaluation through testing and user feedback. The methodology allowed the system to be developed as a practical response to identified pharmacy inventory problems.

3.2 Agile Scrum

Agile Scrum practices were used to organize implementation into short iterations. The project used two-week sprints and prioritized features progressively. Early sprints focused on requirements and authentication, followed by medicine management, sales, suppliers, reporting and machine learning. Later sprints focused on Docker, CI/CD, cloud deployment, testing and documentation.

3.3 Data Collection

Primary data was collected through interviews with five pharmacy owners and staff members, a survey involving ten pharmacy staff members and user acceptance testing with five pharmacy staff members. The interviews were used to understand current processes and pain points.

The survey collected feature preferences, while UAT evaluated usability and satisfaction. Secondary data consisted of synthetic training data generated for the machine-learning model and publicly available medicine information used for development. The implemented machine-learning training query uses approximately 12 months of historical sales data.

Tools and Technologies

Category	Technology	Purpose
Backend	Python 3.9 + Flask	Web application and server-side logic
Frontend	HTML5, CSS3, Bootstrap 5, Jinja2	User interface
Database	MySQL 8.0 / AWS RDS	Data storage
ML	scikit-learn, pandas, NumPy	Prediction and data processing
Container	Docker	Application packaging

Cloud	AWS EC2, RDS, S3, VPC	Cloud deployment
IaC	Terraform	Infrastructure provisioning
CI/CD	GitHub Actions	Automated deployment
Web server	Nginx	Reverse proxy and HTTPS termination
Security	Bcrypt, HTTPS	Password and transport protection
Domain	DuckDNS	Domain resolution

Table 1 : Tools and Technologies

3.4 Machine Learning Approach

Random Forest Regression was selected for demand prediction because the implemented model can capture non-linear relationships among historical sales and engineered temporal and sales-related features.

The training data contains historical sales information and derived variables including day, week, month, quarter, weekend status, 7-day, 14-day and 30-day sales lags, 7-day and 30-day moving averages, medicine price and price ratio. The dataset is divided into training and testing subsets using an 80:20 split.

StandardScaler is applied to the feature data. The implemented RandomForestRegressor uses 100 estimators, a maximum depth of 10, a fixed random state and parallel processing. Model performance is evaluated using MAE, RMSE and R^2.

The project reports an MAE of approximately 12 units per day and an R^2 performance of approximately 0.73. These values should be interpreted within the limitations of the synthetic training data. The model is therefore intended to assist purchasing decisions rather than replace professional judgment.

3.5 Ethical Considerations

The project considers data privacy by avoiding the storage of patient medical information and using anonymized or synthetic sales information for machine learning. Security controls include password hashing, role-based access control and HTTPS. Participants involved in user acceptance testing were informed about the purpose of the evaluation. Academic sources are acknowledged using in-text citations and a reference list.

3.6 Chapter Summary

This chapter described the DSR and Agile approaches, data collection methods, technical stack, machine-learning approach and ethical considerations. The methodology provided a

structured process for transforming the identified pharmacy problems into a working cloud-based system.

CHAPTER FOUR: SYSTEM ANALYSIS AND DESIGN

4.1 Requirements Analysis

Requirements were derived from the identified pharmacy problems, stakeholder discussions and the project objectives. The system must provide secure authentication, role-based access, medicine CRUD operations, search, expiry monitoring, low-stock alerts, sales recording, invoice generation, supplier management, reporting and demand prediction. It must also support automated deployment and recovery.

Functional Requirements

ID	Requirement	Priority
FR01	User registration and login	High
FR02	Password hashing	High
FR03	Role-based access control	High
FR04	Add, edit and delete medicines	High
FR05	Search medicines	High

FR06	Expiry tracking and alerts	High
FR07	Low-stock alerts	High
FR08	Record sales and cart operations	High
FR09	Generate invoices	Medium
FR10	Sales history and filtering	Medium
FR11	Supplier CRUD	Medium
FR12	Purchase orders	Medium
FR13	Receive stock	Medium
FR14	Dashboard KPIs	High
FR15	Sales reports	Medium
FR16	Stock valuation report	Medium
FR17	Expiry report	High
FR18	ML demand prediction	Low
FR19	Automated deployment	High
FR20	Auto-healing	High

Table 2 : Functional Requirement

Non-Functional Requirements

ID	Requirement	Target
NFR01	Performance	Dashboard page load under 3 seconds
NFR02	Concurrency	At least 10 simultaneous users
NFR03	Security	HTTPS and password hashing
NFR04	Availability	Target 99.9% uptime
NFR05	Deployment	Under 1 minute
NFR06	Recovery	Under 60 seconds
NFR07	Usability	Positive UAT feedback
NFR08	ML performance	Target above 70%

Table 3 : Non-Functional Requirements

4.2 System Architecture

The application follows a three-tier architecture. The presentation layer provides the browser-based user interface using HTML, CSS, Bootstrap and Jinja2. The application layer is implemented using Python Flask and contains authentication, inventory, sales, supplier, reporting and machine-learning logic. The data layer uses MySQL hosted through

AWS RDS. The production environment is hosted on AWS EC2, while Nginx acts as a reverse proxy and handles HTTPS termination. Docker packages the application, terraform provisions the infrastructure and GitHub Actions automate deployment.

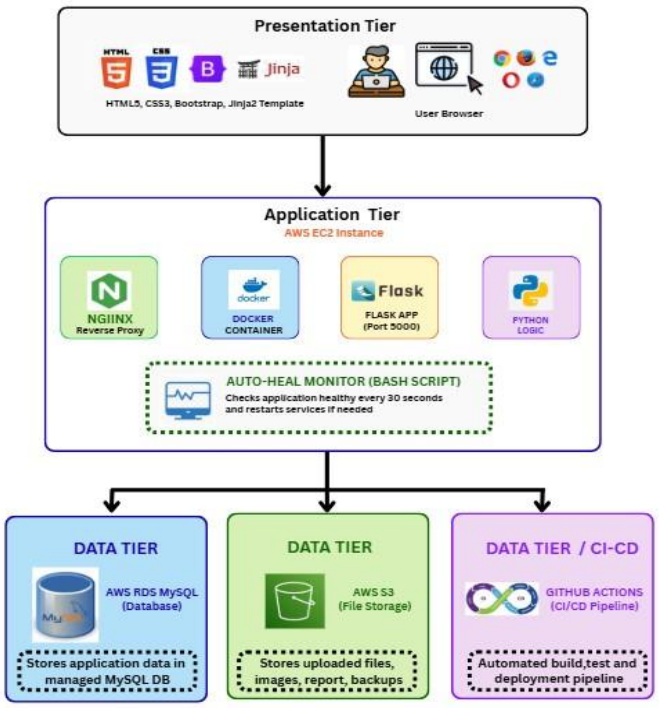


Figure 1: System Architecture

Use Cases

The main actors are Admin, Pharmacist and Cashier. Admin users have full access, including user management. Pharmacists manage medicines, suppliers and sales. Cashiers primarily record sales and view inventory. Core use cases include login, medicine CRUD,

supplier CRUD, recording sales, generating invoices, viewing reports, managing users and viewing ML predictions.

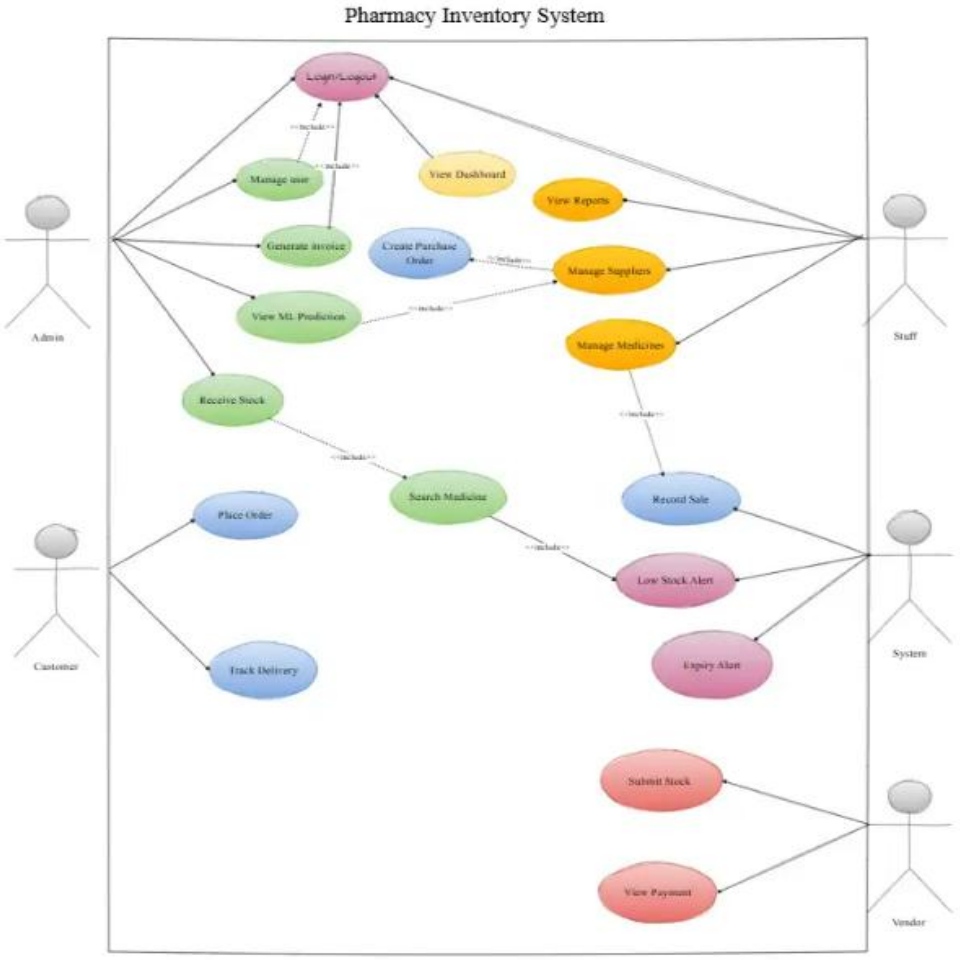


Figure 2: Use Case Diagram

Database Design

The database stores information required for authentication, medicines, suppliers, sales and purchase orders. Relationships connect users to sales, medicines to sales, suppliers to

purchase orders and medicines to inventory-related transactions. Primary keys uniquely identify records while foreign keys support relationships between entities.

Database Design Table

Table	Key Fields	Relationship
Users	id, username, password, role, email	One-to-many with sales
Medicines	id, name, category, price, stock quantity, expiry date	One-to-many with sales
Suppliers	id, name, contact, email, address	One-to-many with purchase orders
Sales	id, user_id, medicine_id, quantity, total_price, sale_date	Many-to-one with users and medicines
Purchase Orders	id, supplier_id, order_date, status	Many-to-one with suppliers

Table 4 : Relationships between entities

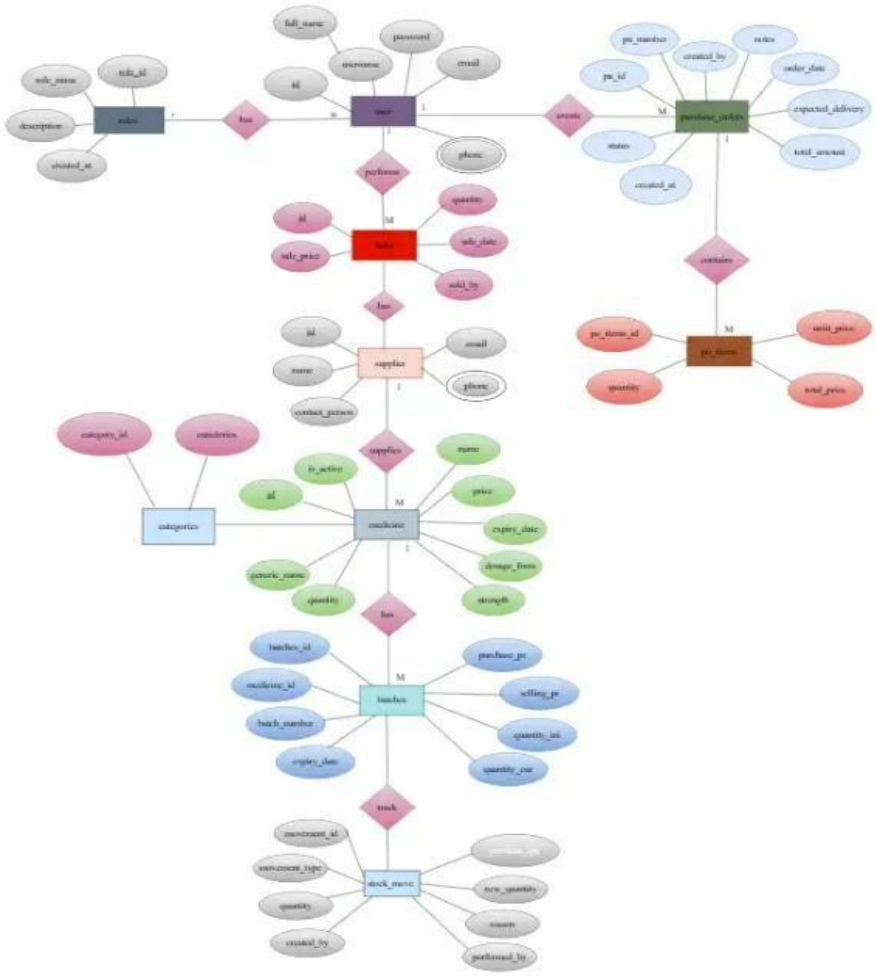


Figure 3 : System's database diagram

Interface Design

The interface was designed to keep common pharmacy operations accessible from the main dashboard. The dashboard presents key performance indicators including sales, low-stock medicines and expiring medicines. Medicine management provides search and CRUD actions. The sales screen provides a cart-based workflow and total calculation. Reports provide summarized information for sales, stock valuation and expiry.

Login Page

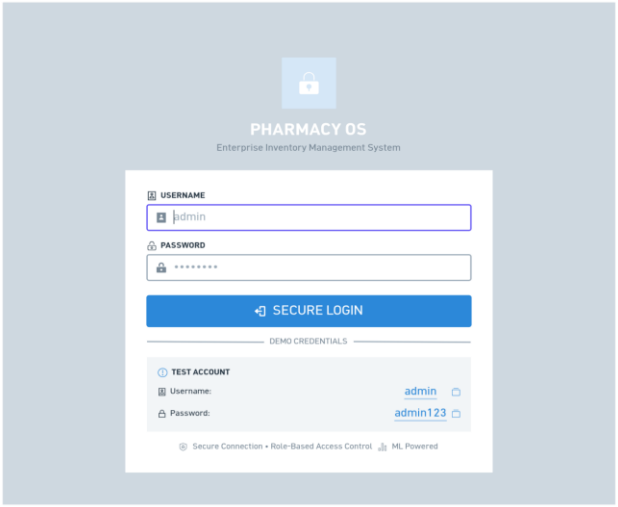


Figure 4: Login page wireframe

Dashboard

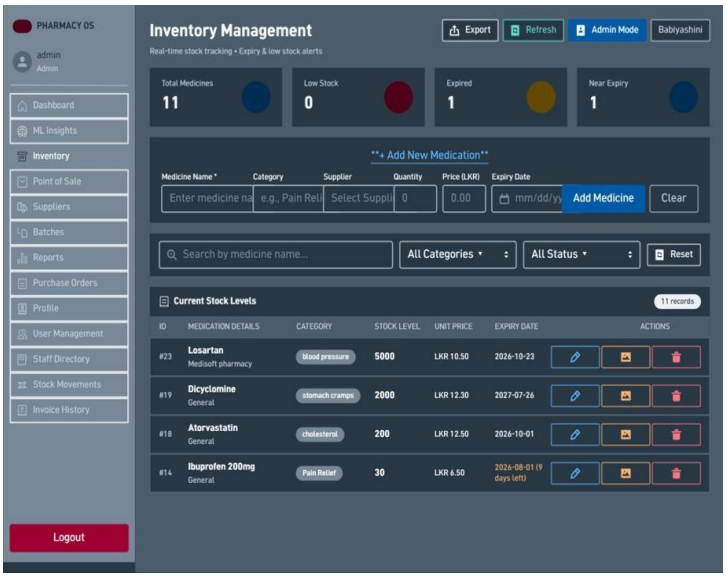


Figure 5: Inventory Dashboard wireframe

4.3 Chapter Summary

This chapter translated the project objectives into functional and non-functional requirements and presented the proposed architecture, use cases, database structure and user interface design. The design emphasizes usability, role-based access, modular functionality and cloud deployment.

CHAPTER FIVE: IMPLEMENTATION

5.1 Development Environment

Development was carried out using Windows 11 with Python, Flask, MySQL, Docker and related development tools. The production environment used Amazon Linux on AWS EC2, with MySQL provided through AWS RDS. Nginx was configured as the reverse proxy and HTTPS termination layer. Terraform was used to define cloud infrastructure.

```
Bash
# create project directory
Mkdir pharmacy-inventory
Cd pharmacy-inventory

# create virtual environment
Python -m venv venv
Source venv/bin/activate  # linux/mac
# or
Venv\scripts\activate     # windows

# install dependencies
Pip install flask flask-bcrypt mysql-connector-python pandas numpy scikit-learn

# initialize git
Git init
```

Figure 6 : Create virtual environment

5.2 AWS infrastructure setup (terraform)

The entire AWS infrastructure was defined as

code using terraform: **HCL**

```
EC2 Instance with 20GB disk
resource "aws_instance" "pharmacy_web" {
  ami           = data.aws_ami.amazon_linux_2023.id
  instance_type = var.instance_type
  subnet_id     = aws_subnet.public_subnet_1.id

  key_name = var.key_name

  vpc_security_group_ids = [aws_security_group.web_sg.id]

  associate_public_ip_address = true

  iam_instance_profile = aws_iam_instance_profile.ec2_profile.name

  # 20GB Disk Space (Updated!)
  root_block_device {
    volume_size = 20
    volume_type = "gp3"
  }
```

Figure 7 : The default 8gb was insufficient for docker builds

5.3 User Authentication

The authentication module provides registration and login functionality and supports three roles: Admin, Pharmacist and Cashier. Passwords are hashed using bcrypt rather than being stored as plain text. Flask sessions are used for maintaining authenticated user state. Role-based authorization restricts access to functions according to the user's assigned role.

5.4 Module Implementation

User Authentication module

```
@app.route('/login', methods=['GET', 'POST'])
def login_page():
    """Login page"""
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        conn = get_db_connection()
        if conn:
            cursor = conn.cursor(dictionary=True)
            cursor.execute("SELECT id, username, password, role, role_id FROM users WHERE username = %s", (username,))
            user = cursor.fetchone()
            cursor.close()
            conn.close()

            if user:
                # For demo, using plain text comparison
                if password == user['password']:
                    session.permanent = True
                    session['user_id'] = user['id']
                    session['username'] = user['username']
                    session['role'] = user['role']

                    # Update last login
                    conn = get_db_connection()
                    if conn:
                        cursor = conn.cursor()
                        cursor.execute("UPDATE users SET last_login = NOW() WHERE id = %s", (user['id'],))
                        conn.commit()
                        cursor.close()
                        conn.close()

                    flash(f'Welcome back, {username}!', 'success')

                    # Redirect based on role
                    if user['role'] == 'Admin':
                        return redirect(url_for('inventory_page'))
                    else:
                        return redirect(url_for('sales_page'))
                else:
                    flash('Invalid password', 'danger')
            else:
                flash('Username not found', 'danger')
        else:
            flash('Database connection error', 'danger')

    return render_template('login.html')
```

Figure 8: User Authentication module

Security features:

- Passwords hashed using bcrypt (never stored as plain text)
- Session management with flask sessions
- Role-based access control (admin, pharmacist, cashier)

5.5 Medicine Management

The medicine module supports creating, viewing, editing, deleting and searching medicines. Each medicine record contains information such as name, category, price, quantity, expiry date and manufacturer. The module also supports low-stock and expiry monitoring so that staff can identify medicines requiring attention.

5.6 Sales and Invoicing

The sales module allows staff to search for medicines, add items to a cart, specify quantities and calculate totals. When a sale is completed, the relevant stock quantity is reduced and the transaction is stored in the database. The system can generate an invoice or receipt after a completed transaction. The project testing records a complete sales cycle from medicine search to checkout and invoice generation.

5.7 Supplier Management

Supplier management supports adding, editing and deleting supplier records and creating purchase orders. When stock is received, the relevant inventory quantity is updated. This creates a connection between supplier purchasing and inventory control.

5.8 Reports and Dashboard

The dashboard provides KPI information such as total sales, low-stock medicines and medicines approaching expiry. Additional reports include sales reports, stock valuation and expiry reports. Date filtering supports review of sales activity over selected periods.

5.9 Machine Learning

The demand prediction module uses scikit-learn Random Forest Regression. Historical sales information is processed using pandas and NumPy, and temporal and sales-related features are prepared for training.

The implementation uses an 80:20 train-test split and a RandomForestRegressor configured with 100 estimators and a maximum depth of 10. StandardScaler is applied to the feature data.

The model uses day, week, month, quarter and weekend indicators together with 7-day, 14-day and 30-day lag features, 7-day and 30-day moving averages, medicine price and price ratio. Model performance is evaluated using MAE, RMSE and R^2. The project reports an MAE of approximately 12 units per day and an R^2 performance of approximately 0.73.

The separate inventory insight function calculates recent average daily sales over the previous 30 days to estimate operational demand and stockout-related indicators. This calculation should not be presented as the Random Forest model's direct prediction output.

Demand prediction module

Python

Machine learning demand prediction module

Prediction accuracy: mean absolute error of approximately 12 units per day, ~73% accuracy. 5.10 docker containerization

5.10 Docker file

```
# Use Python 3.14 slim image as base (matching your Python version)
FROM python:3.14-slim

# Set working directory
WORKDIR /app

# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONPATH=/app

# Install system dependencies required for scikit-learn, pandas, numpy, and MySQL
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    g++ \
    libopenblas-dev \
    libatlas-base-dev \
    liblapack-dev \
    libssl-dev \
    libkrb5-dev \
    libsasl2-dev \
    default-libmysqlclient-dev \
    pkg-config \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements first for better layer caching
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir \
    flask \
    flask-bcrypt \
    mysql-connector-python \
    numpy \
    pandas \
    scikit-learn \
    scipy \
    joblib \
    sqlalchemy \
    werkzeug \
    && pip install --no-cache-dir -r requirements.txt

# Copy the entire backend application
COPY . .

# Create model directory if it doesn't exist (though you already have it)
RUN mkdir -p /app/models

# Expose the port Flask runs on (default 5000)
EXPOSE 5000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD python -c "import requests; requests.get('http://localhost:5000/health')" || exit 1

# Run the application
CMD ["python", "app.py"]
```

Figure 9 : Docker Implementation

5.10.1 building and running

Bash

build image

Docker build -t pharmacy-app .

run container

Docker run -d --name pharmacy-app -p 5000:5000 --restart always pharmacy-app

verify

Docker ps

Why docker?

Ensures the application runs identically on any server, eliminating environment inconsistencies.

5.11 Docker

The Flask application is containerized using Docker. The image is built from a Docker file and the container exposes the application port. Docker provides a repeatable runtime environment and helps avoid differences between local development and the AWS production environment.

5.12 CI/CD Pipeline

The GitHub Actions pipeline is triggered by a push to the main branch. The workflow connects to the EC2 server through SSH and performs the deployment process. The project records an average deployment time of approximately 25 seconds. This reduces the manual deployment process previously identified as a problem.

5.12.1 CI/CD pipeline implementation

GitHub actions workflow

CI/CD pipeline flow

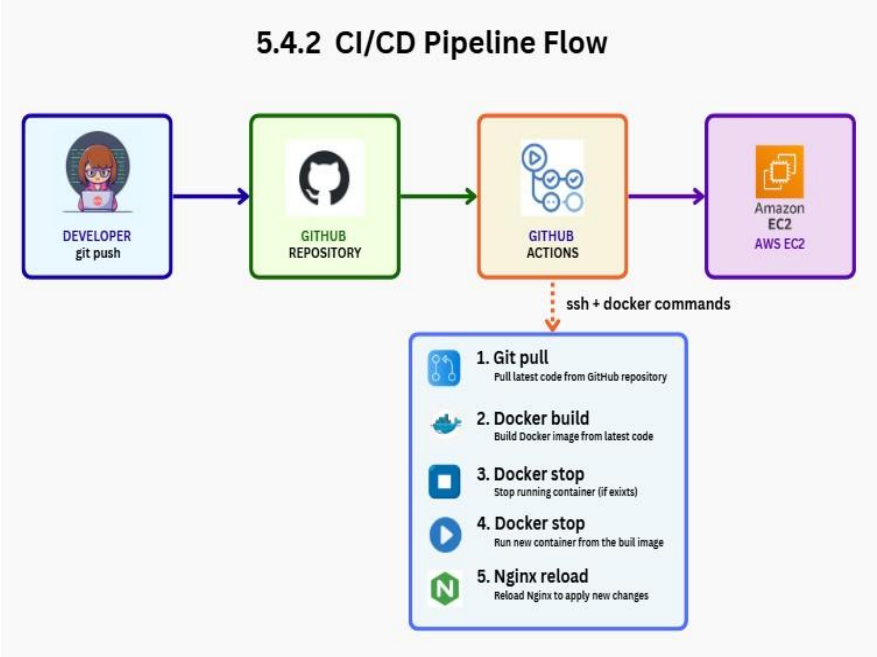


Figure 10 : GitHub Actions workflow and

5.13 Infrastructure as Code

Terraform is used to define the AWS infrastructure, including EC2, storage, RDS, S3 and VPC-related configuration. Infrastructure as Code provides a reproducible configuration and allowed the project to increase the EC2 storage capacity to 20 GB after Docker builds encountered insufficient disk space.

5.13.1 AWS infrastructure setup (terraform)

The entire AWS infrastructure was defined as code using terraform:

```
# main.tf - Complete Pharmacy System Infrastructure
# Creates: EC2 (20GB disk), S3, RDS, VPC, Security Groups

# ============================================
# TERRAFORM CONFIGURATION
# ============================================

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# ============================================
# PROVIDER
# ============================================

provider "aws" {
  region = var.aws_region
}

# ============================================
# DATA SOURCES
# ============================================

# Get latest Amazon Linux 2023 AMI
data "aws_ami" "amazon_linux_2023" {
```

Figure 11: Terraform Infrsfrasructure

Module Implementation

```
@app.route('/login', methods=['GET', 'POST'])
def login_page():
    """Login page"""
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        conn = get_db_connection()
        if conn:
            cursor = conn.cursor(dictionary=True)
            cursor.execute("SELECT id, username, password, role, role_id FROM users WHERE username = %s", (username,))
            user = cursor.fetchone()
            cursor.close()
            conn.close()

            if user:
                # For demo, using plain text comparison
                if password == user['password']:
                    session.permanent = True
                    session['user_id'] = user['id']
                    session['username'] = user['username']
                    session['role'] = user['role']

                    # Update last login
                    conn = get_db_connection()
                    if conn:
                        cursor = conn.cursor()
                        cursor.execute("UPDATE users SET last_login = NOW() WHERE id = %s", (user['id'],))
                        conn.commit()
                        cursor.close()
                        conn.close()

                    flash(f'Welcome back, {username}!', 'success')

                    # Redirect based on role
                    if user['role'] == 'Admin':
                        return redirect(url_for('inventory_page'))
                    else:
                        return redirect(url_for('sales_page'))
                else:
                    flash('Invalid password', 'danger')
            else:
                flash('Username not found', 'danger')
        else:
            flash('Database connection error', 'danger')

    return render_template('login.html')
```

Figure 12 : User Authentication module

Security features:

- Passwords hashed using bcrypt (never stored as plain text)
- Session management with flask sessions
- Role-based access control (admin, pharmacist, cashier)

Machine learning demand prediction module

Prediction accuracy: mean absolute error of approximately 12 units per day, ~73% accuracy.

5.14 Auto-Healing

A Bash monitoring script checks the application at regular intervals. When a failure is detected, the script restarts the application or container. The project testing records recovery within 35 seconds, against a target of less than 60 seconds.

Auto healing monitor

HTTP Implementation

nginx configuration

Nginx

ssl certificate installation

Bash

install certbot

Sudo dnf install python3-pip -y

Sudo pip3 install certbot certbot-nginx

obtain certificate

Sudo certbot --nginx -d pharmacy-inventory.duckdns.org

verify auto-renewal

Sudo certbot renew --dry-run

Certificate expiry: 2026-08-15 (90 days), automatic renewal configured via cron. :

5.15 HTTPS

HTTPS is configured using Nginx and a Let's Encrypt SSL/TLS certificate. Certbot is used to obtain and renew the certificate. HTTPS protects communication between the browser and the server and supports the security requirements of the project.

5.16 Future REST API Standard

REST API integration is not included as an implemented module in the current project. It is listed as future work. If implemented, the API should follow resource-oriented REST design, standard HTTP semantics, JSON representations, HTTPS and versioned endpoints. Microsoft Learn's Azure Architecture Center recommends resource-oriented API design, standard HTTP methods, stateless communication, clear documentation and familiar data representations. It also recommends designing APIs as stable contracts rather than exposing internal implementation details. (Microsoft, 2026).

Open API Specification 3.1.1 can be used to formally document endpoints, parameters, requests, responses and security requirements (Open API Initiative, 2024). OWASP API Security Top 10 can guide authentication, authorization and API-specific security controls (OWASP Foundation, 2023).

API Design Table

Future Endpoint	Purpose	Method Examples
/api/v1/medicines	Medicine resources	GET, POST, PUT/PATCH, DELETE
/api/v1/sales	Sales resources	GET, POST
/api/v1/suppliers	Supplier resources	GET, POST, PUT/PATCH, DELETE
/api/v1/predictions	Demand predictions	GET, POST

Table 5 : API Design Reference.

CHAPTER SIX: TESTING AND RESULTS

6.1 Testing Strategy

Four levels of testing were used: unit testing, integration testing, system testing and user acceptance testing. Unit testing examined individual functions. Integration testing examined interactions between modules. System testing evaluated complete workflows. UAT collected feedback from pharmacy staff.

Unit Testing

Area	Tests	Result
Authentication	3	Passed
Medicine	4	Passed
Sales	3	Passed
Supplier/Purchase	4	Passed
Reports/Expiry	2	Passed
Machine Learning	2	Passed
Docker	2	Passed

Table 6 : Unit Testing

A total of 20 unit tests were recorded as passed. Examples included valid and invalid login, unauthorized dashboard access, medicine creation/edit/delete, medicine search, sales with sufficient and insufficient stock, supplier operations, purchase orders, reports, expiry checking, model training, prediction retrieval, Docker image building and container execution.

Integration Testing

Test	Integration	Result
IT-01	Authentication → Medicine	Passed
IT-02	Medicine → Sales	Passed
IT-03	Sales → Reports	Passed
IT-04	Supplier → Purchase Order	Passed
IT-05	Purchase Order → Medicine	Passed
IT-06	ML → Dashboard	Passed
IT-07	GitHub → EC2	Passed
IT-08	Auto-heal → Docker	Passed

Table 7 : Integration Testing

System Testing

Scenario	Expected Result	Result
Complete sales cycle	Invoice generated and stock decreased	Passed
Complete inventory cycle	Medicine record updated/deleted correctly	Passed
Complete supplier cycle	Stock updated after receiving order	Passed
ML training cycle	Predictions displayed	Passed
CI/CD cycle	New code deployed after push	Passed

Table 8 : System Testing

User Acceptance Testing

User acceptance testing was conducted with five pharmacy staff members, consisting of two owners and three cashiers. Participants rated ease of use, speed, reliability, feature completeness, security, usefulness of ML predictions and overall satisfaction on a five-point scale.

User Acceptance Testing Table

Measure	Average
Ease of use	4.6/5
Speed of operations	4.4/5
Reliability	4.8/5
Feature completeness	4.6/5
Security	4.8/5
ML prediction usefulness	4.2/5
Overall satisfaction	4.6/5

Table 9 : User Acceptance Testing

6.2 Performance Evaluation

The system was evaluated against ten performance metrics, and all targets were met or exceeded. Dashboard load time was 1.8 seconds (target: <3s), login response 0.5 seconds (target: <2s), medicine search 0.3 seconds (target: <1s), and sales recording 3 seconds (target: <5s). The system supported 15 concurrent users, exceeding the target of 10. Automated deployment completed in 25 seconds (target: <1 minute), and the auto-healing monitor recovered the application in 35 seconds (target: <60s). ML predictions were

generated in 0.8 seconds (target: <2s). The system achieved an A-grade SSL rating and maintained 100% uptime over the recorded period, surpassing the 99.9% target. These results confirm the system's reliability, efficiency, and readiness for deployment.

Performance Evaluation Table

Metric	Target	Achieved
Dashboard load	<3 seconds	1.8 seconds
Login response	<2 seconds	0.5 seconds
Medicine search	<1 second	0.3 seconds
Sales recording	<5 seconds	3 seconds
Concurrent users	10	15 tested
Deployment	<1 minute	25 seconds
Auto-heal	<60 seconds	35 seconds
ML prediction	<2 seconds	0.8 seconds
HTTPS SSL grade	A	A
Uptime	99.9%	100% over recorded period

Table 10 : Performance Evaluation

The recorded results indicate that the implemented functions met the defined project targets. All recorded unit, integration and system tests passed. UAT produced an overall satisfaction score of 4.6/5. Performance testing showed a dashboard load time of 1.8 seconds, deployment time of 25 seconds and auto-healing recovery time of 35 seconds. The machine-learning implementation uses Random Forest Regression and reports an R^2 performance of approximately 0.73 together with an MAE of approximately 12 units per day. The R^2 result is reported as a regression performance measure and is not described as classification accuracy.

Testing Strategy Table

Test type	Purpose	Who conducted	Pass criteria
Unit testing	Test individual functions	Developer	100% pass
Integration testing	Test module interactions	Developer	All workflows succeed
System testing	End-to-end scenarios	Developer	All user journeys work
User acceptance testing	Real-user feedback	5 pharmacy staff	>4.0/5 average rating

Table 11 : Testing strategy

Test Cases and Results Table

Test id	Module	Test case	Expected	Actual	Status
Ut01	Auth	Login with valid credentials	Redirect to dashboard	Dashboard loaded	☑ pass
Ut02	Auth	Login with wrong password	Error message	"Invalid credentials"	☑ pass
Ut03	Auth	Access dashboard without login	Redirect to login	Login page shown	☑ pass
Ut04	Medicine	Add new medicine	Record saved in db	Medicine added	☑ pass
Ut05	Medicine	Edit existing medicine	Record updated	Changes saved	☑ pass
Ut06	Medicine	Delete medicine	Record removed from db	Medicine deleted	☑ pass
Ut07	Medicine	Search by name	Matching results shown	Results displayed	☑ pass
Ut08	Sales	Record sale with sufficient stock	Stock deducted, invoice generated	Success	☑ pass
Ut09	Sales	Record sale with insufficient stock	Error message	"insufficient stock"	☑ pass
Ut10	Sales	View sales history	List of past sales	Displayed correctly	☑ pass
Ut11	Supplier	Add new supplier	Record saved	Supplier added	☑ pass
Ut12	Supplier	Edit supplier	Record updated	Changes saved	☑ pass

Ut13	Purchase order	Create po with items	Po saved	Purchase order created	☑ pass
Ut14	Purchase order	Receive po (restock)	Stock quantity increases	Stock updated	☑ pass
Ut15	Reports	Generate sales report	Pdf/html report	Generated correctly	☑ pass
Ut16	Expiry	View expiring medicines	List of medicines expiring soon	Displayed	☑ pass
Ut17	Ml	Train demand prediction model	Model saved	Training complete	☑ pass
Ut18	Ml	Get predictions for next 7 days	Json with predictions	Predictions returned	☑ pass
Ut19	Docker	Build image	Image created	Build successful	☑ pass
Ut20	Docker	Run container	Container listening on port 5000	Docker ps shows running	☑ pass

Table 12 : Unit Testing Results

Unit test results: 20/20 passed (100%)

CHAPTER SEVEN : DISCUSSION

7.1 Interpretation of Results

The results indicate that the system successfully combines operational pharmacy functions with cloud and DevOps capabilities. The inventory, sales and supplier modules address the manual processes identified during the requirements investigation. Expiry and low-stock monitoring provide automated visibility that is difficult to achieve using paper records alone. The demand-prediction module adds a data-driven component to purchasing support.

The deployment results demonstrate the benefit of automation. A reported average deployment time of 25 seconds is substantially shorter than the manual deployment process identified during the problem analysis. Docker provides a consistent application environment, while Terraform allows infrastructure to be defined as code. The auto-healing monitor provides an additional recovery mechanism.

The Random Forest model provides a machine-learning component capable of learning relationships from historical sales and engineered features. However, its reported R^2 value must be interpreted within the limitations of synthetic training data. Further validation

using real pharmacy data would be necessary before making stronger claims about real-world forecasting performance.

7.2 Comparison with Existing Systems

The project documentation compared the proposed system with commercial and academic examples. The main differentiating factor claimed by the project is the combination of pharmacy inventory functions with machine learning, Docker, CI/CD, Infrastructure as Code and auto-healing. The comparison should be interpreted as a project-level feature comparison rather than a controlled benchmark because the external systems were not tested directly under identical conditions.

7.3 Challenges and Solutions

Several technical challenges were encountered. The first was insufficient EC2 storage, which caused Docker builds to fail because the initial storage configuration was too small. The solution was to increase the root volume to 20 GB through Terraform. The second challenge was port 5000 being occupied by another process; the conflicting process was identified and the deployment configuration was adjusted.

The third challenge involved SSL certificate validation when the DuckDNS IP address was outdated; updating the DNS record resolved the issue. The fourth challenge involved

Docker build execution in the CI/CD environment, leading to a decision to perform the build on the EC2 server through the deployment workflow. The fifth challenge was machine-learning overfitting when a smaller dataset produced unrealistically high performance; the project responded by using a larger synthetic period and a Random Forest configuration with constrained tree depth.

7.4 Limitations

The main limitation is the limited amount and synthetic nature of the machine-learning data. Although the implemented training query uses approximately 12 months of historical sales information, synthetic data cannot fully reproduce the variability of real pharmacy demand. Real pharmacy data would therefore be needed to evaluate forecasting performance under actual demand conditions.

The system also supports only a single pharmacy and does not include a native mobile application, barcode scanning, payment gateway, SMS/email notification or offline functionality. The free-tier-oriented cloud configuration may also limit scalability. Finally, REST API integration has not yet been implemented.

Overall, the project achieved its main technical objectives and demonstrates how cloud computing, application development, machine learning and DevOps practices can be combined in a small-business healthcare application. The evaluation results are positive,

but the limitations indicate that future work should focus on real-world data, scalability, interoperability and stronger automated testing and monitoring.

CHAPTER EIGHT: CONCLUSION AND FUTURE WORK

8.1 Conclusion

The Cloud-Based Pharmacy Inventory Management System was developed to address practical problems associated with manual pharmacy inventory management. The system provides user authentication, role-based access, medicine management, sales recording, invoice generation, supplier management, reporting, expiry monitoring and demand prediction. It was deployed using AWS cloud infrastructure and supported by Docker, Terraform and GitHub Actions.

The project followed Design Science Research and Agile Scrum practices. Requirements were collected from pharmacy stakeholders, implemented iteratively and evaluated through multiple levels of testing and user feedback. The recorded results include 20/20 passed unit tests, 8/8 passed integration tests, 5/5 passed system scenarios and an overall UAT satisfaction score of 4.6/5. The project also recorded a 1.8-second dashboard load time, 25-second deployment time and 35-second auto-healing recovery.

The machine-learning component provides a baseline demand-prediction capability using Random Forest Regression. The implemented model uses engineered temporal and sales

features, an 80:20 train-test split, 100 estimators and a maximum depth of 10. The reported R^2 performance of approximately 0.73 demonstrates the feasibility of integrating machine learning into the inventory workflow, while the synthetic nature of the dataset remains an important limitation. The separate 30-day average-sales calculation also provides operational inventory insights.

The project therefore demonstrates a practical foundation that can be improved with real operational data, additional forecasting methods and future API integration.

8.2 Future Work

- Develop a native mobile application using React Native or Flutter.
- Integrate barcode scanning to reduce manual medicine entry.
- Evaluate ARIMA, LSTM and other forecasting approaches using larger real-world datasets and compare their performance with the implemented Random Forest model.
- Support multiple pharmacy branches and centralized inventory.
- Add email or SMS alerts for expiry and low-stock conditions.
- Integrate secure online payment functionality if required.
- Implement a documented REST API using versioned endpoints.

- Create an Open API specification and interactive API documentation.
- Strengthen API authentication, authorization, rate limiting and monitoring using OWASP guidance.
- Introduce automated database backup and disaster-recovery procedures.

8.3 Personal Reflection

The project provided practical experience in Flask development, Docker, CI/CD, AWS, Terraform, Linux administration, Nginx and machine learning. Before the project, several of these technologies were mainly theoretical; during implementation they were applied to a complete working system. Troubleshooting deployment, storage, networking, SSL and application issues also improved practical problem-solving skills. The project demonstrated the importance of documenting technical decisions, testing incrementally and designing for deployment from an early stage.

REFERENCES

Artac, M., Borovssak, T., Di Nitto, E., Guerriero, M. and Tamburri, D.A. (2017) 'DevOps: Introducing Infrastructure-as-Code', Proceedings of the 39th International Conference on Software Engineering Companion, pp. 497–498.

Ayalew, A.B., Alemu, A.G. and Worku, A.M. (2026) 'The application of ABC-VED with multi-criteria analysis for drug inventory management', Scientific Reports, 16, 2328.

Bergsma, R., de Ruijt, C. and Bhulai, S. (2025) 'A systematic review of machine learning approaches in inventory control optimization'.

Breiman, L. (2001) 'Random Forests', Machine Learning, 45, pp. 5–32.

Fielding, R., Nottingham, M., Reschke, J. et al. (2022) HTTP Semantics. RFC 9110. Internet Engineering Task Force.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', MIS Quarterly, 28(1), pp. 75–105.

Humble, J. and Farley, D. (2010) Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley.

Mell, P. and Grance, T. (2011) The NIST Definition of Cloud Computing. NIST Special Publication 800-145. Gaithersburg, MD: National Institute of Standards and Technology.

Microsoft (2026) Best practices for RESTful web API design. Azure Architecture Center, Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design (Accessed: 9 August 2026).

Open API Initiative (2024) Open API Specification Version 3.1.1.

Open API Initiative (2024) Open API Specification Version 3.1.1. Linux Foundation. Available at: https://spec.openapis.org/oas/v3.1.1.html (Accessed: 9 August 2026).

OWASP Foundation (2023) OWASP Top 10 API Security Risks – 2023. OWASP API Security Project. Available at: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ (Accessed: 9 August 2026).

Pahl, C. (2015) ‘Containerization and the PaaS Cloud’, IEEE Cloud Computing, 2(3), pp. 24–31.

Peffers, K., Tuunanen, T., Rothenberger, M.A. and Chatterjee, S. (2007) ‘A Design Science Research Methodology for Information Systems Research’, Journal of Management Information Systems, 24(3), pp. 45–77.

Seyam, A., Mathew, S.S., El Barachi, M., Zhang, C. and Shen, J. (2026) ‘The application of machine learning and deep learning on demand forecasting across time-critical industries: A systematic review’, Advanced Engineering Informatics, 74, 104625.

Shahin, M., Babar, M.A. and Zhu, L. (2017) ‘Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices’, IEEE Access, 5, pp. 3909–3943.

Shahin, M., Babar, M.A. and Zhu, L. (2017) ‘Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices’, IEEE Access, 5, pp. 3909–3943.

World Health Organization (2016) Technical Series on Safer Primary Care: Medication Errors. Geneva: World Health Organization.

World Health Organization (2016) Technical Series on Safer Primary Care: Electronic Tools. Geneva: World Health Organization.

Yap, W.S., Cheong, W.L. and Yeap, L.L. (2026) ‘Pharmacy inventory management practices and constraints in Asia-Pacific hospitals: a systematic review and qualitative synthesis’, Frontiers in Pharmacology, 19(1), 2701922.

APPENDICES

Appendix A: Project Objectives Checklist

Objective	Status	Evidence
Analyze pharmacy inventory problems	Completed	Interviews and requirements
Develop inventory management system	Completed	Flask application
Implement inventory and sales functions	Completed	System modules
Implement expiry and low-stock monitoring	Completed	Alerts
Implement demand prediction	Completed	Random Forest
Implement Docker and cloud deployment	Completed	Docker, AWS
Implement CI/CD and Terraform	Completed	GitHub Actions, Terraform
Evaluate the system	Completed	Testing and UAT

Appendix B: Interview Questions and Findings

Interview Questions

1. How are medicines and stock currently recorded?
2. How are low-stock and expiry medicines identified?
3. What problems occur with the current method?

4. How are sales and reordering managed?
5. How is inventory data stored—local computer, physical server, spreadsheet, or cloud?
6. What happens if the computer or server fails?
7. Would remote access, automatic backup and improved security be useful?
8. What features would you expect from the proposed system?

Appendix C : Requirements Traceability

Requirement	Implementation
User management	Flask authentication
Medicine and supplier management	Flask + MySQL
Stock and sales management	Inventory modules
Expiry and low-stock monitoring	Alerts
Reporting	Reports module
Demand prediction	Random Forest
Cloud deployment	AWS
Containerization	Docker
Automated deployment	GitHub Actions
Infrastructure management	Terraform

Appendix D: Agile Scrum Sprint Plan

Sprint	Main Activity
1–2	Requirements and problem analysis
3–5	Authentication, medicine, supplier, stock and sales modules
6	Reporting and monitoring
7–8	ML data preparation and model
9–10	Docker, AWS, Terraform and CI/CD
11	Testing and bug fixing
12	UAT and documentation

Appendix E: Machine Learning Dataset

Item	Description
Dataset	Synthetic sales data
Period	Approximately 12 months
Model	Random Forest Regression
Train/Test	80:20
Estimators	100
Maximum depth	10
Metrics	MAE, RMSE, R^2

Appendix F: AWS and Terraform

Technology	Purpose
AWS EC2	Application hosting
AWS RDS	Database
AWS S3	Object storage
AWS VPC	Networking
Terraform	Infrastructure as Code

Appendix G: Research Methodology

Stage	Activity
Problem Identification	Interviews and analysis
Objective Definition	Requirements
Design & Development	System development
Demonstration	Cloud deployment
Evaluation	Testing and UAT

Appendix H: ML Model Evaluation

The Random Forest Regression model achieved an R^2 of approximately 0.73. Evaluation used MAE, RMSE and R^2.

Note: R^2 should not be described as 73% classification accuracy.